

Base de Datos de un Spa-Canino

MATERIA: BASE DE DATOS I

DOCENTE: MARIN SALAZAR CARMEN ROSA

PROYECTO: SISTEMA DE GESTIÓN DE SPA-CANINO

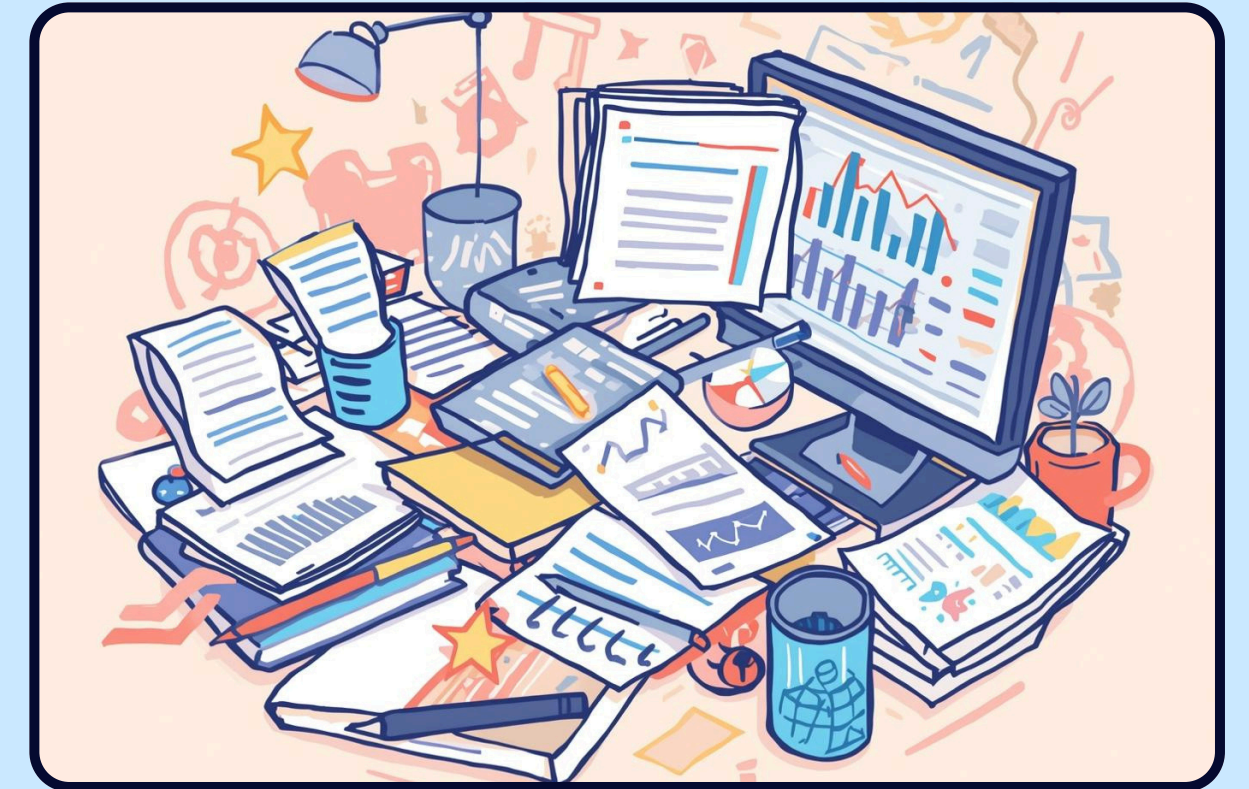
INTEGRANTES: BELÉN MEJÍA MEDINA

PAOLA ROSENDA QUINTEROS PÉREZ

Introducción al Proyecto

DESCRIPCIÓN DEL PROBLEMA

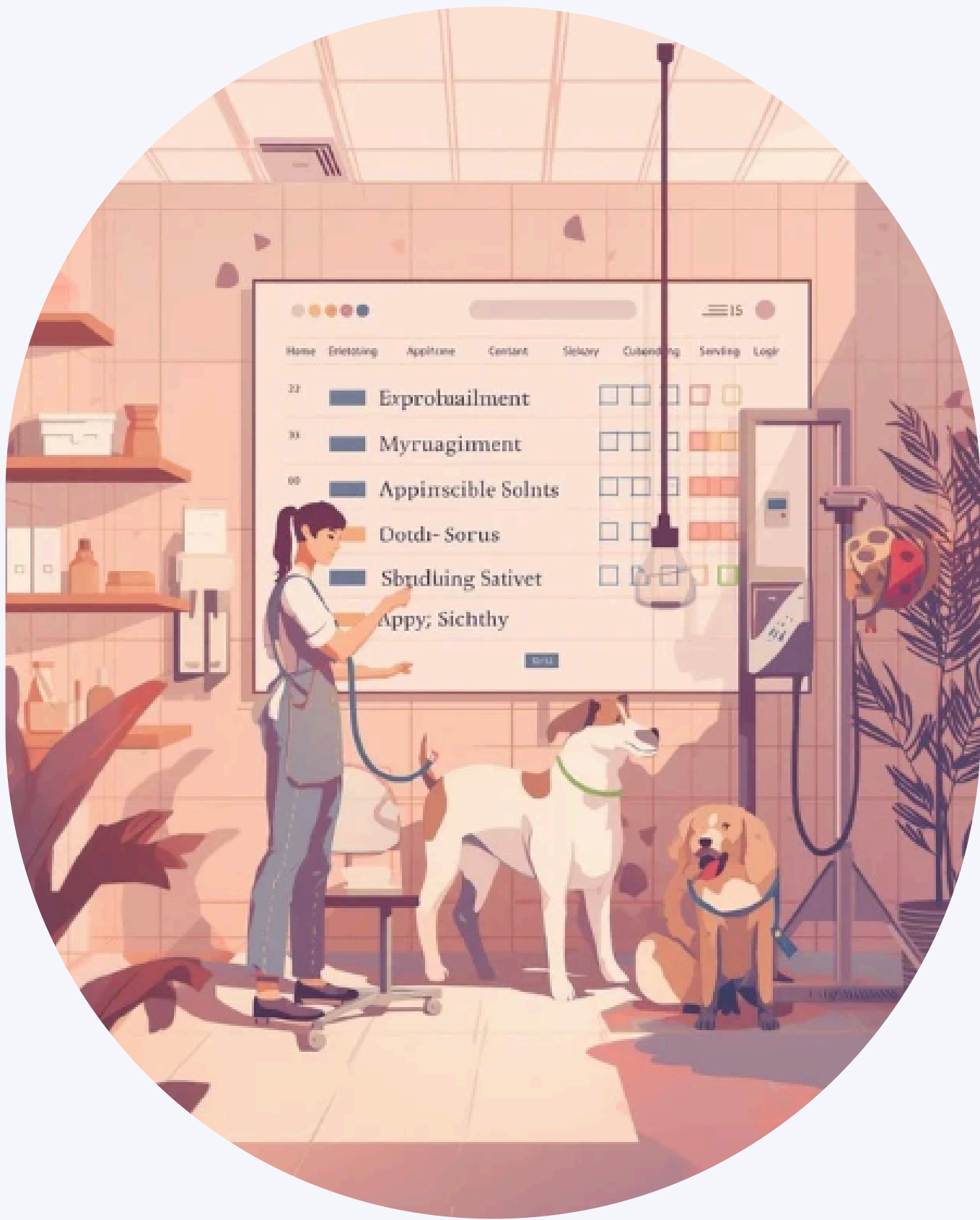
El Spa-Canino enfrenta desafíos en la gestión de clientes, mascotas y citas, con errores manuales que causan duplicación, pérdida de información y conflictos de horarios.



PROPUESTA DE SOLUCIÓN

Un sistema de base de datos relacional para gestionar tiempos, asignación de groomers y facturación, centrado en la eficiencia operativa.





Problemas Actuales en la Gestión

Descontrol de Agenda

La supervisión manual de citas genera tiempos muertos o sobrecarga para los groomers, afectando la calidad estética.

Pérdida de Información para la Seguridad

Desconocimiento del temperamento de las mascotas (agresivo, nervioso) aumenta el riesgo de accidentes durante el servicio.

Características de la Solución

Registro Detallado

Clientes y Macotas, con énfasis en el comportamiento.

Gestión de Citas

Registro de citas con validacion de fecha futura y estado.

Catálogo de Servicios

Servicios con precios fijos y tiempos estimados.

Estado de Cita

Desde reserva (pendiente) hasta finalización (atendida).



Reglas de Negocio Clave

1. Teléfonos Únicos

Los números telefónicos de clientes deben ser únicos para evitar inconvenientes.

2. Fechas de Nacimiento Válidas

Las fechas de nacimiento de las mascotas no deben ser futuras.

3. Temperamento Obligatorio

El temperamento de la mascota debe registrarse para seguridad operativa.

4. Mascota Asociada a Cliente

Cada mascota debe estar asociada a un cliente para contacto.

5. Cita con Mascota Asignada

Una cita debe tener una mascota asignada para integridad de datos.

7. Agenda de Groomers

Los groomers no deben tener citas solapadas en estado Pendiente o Confirmada.

8. Groomer Responsable

Cada cita debe asignar a un solo groomer responsable.

9. Tarifa Fija

La tarifa de los servicios es fija para coherencia financiera.

10. Factura Única por Cita

Una ID_Cita solo puede generar una factura.

11. No Pago por Citas Canceladas

Una cita cancelada no debe generar pago.

12. Monto Total Positivo

El monto total a facturar debe ser mayor a 0.

13. Método de Pago Estandarizado

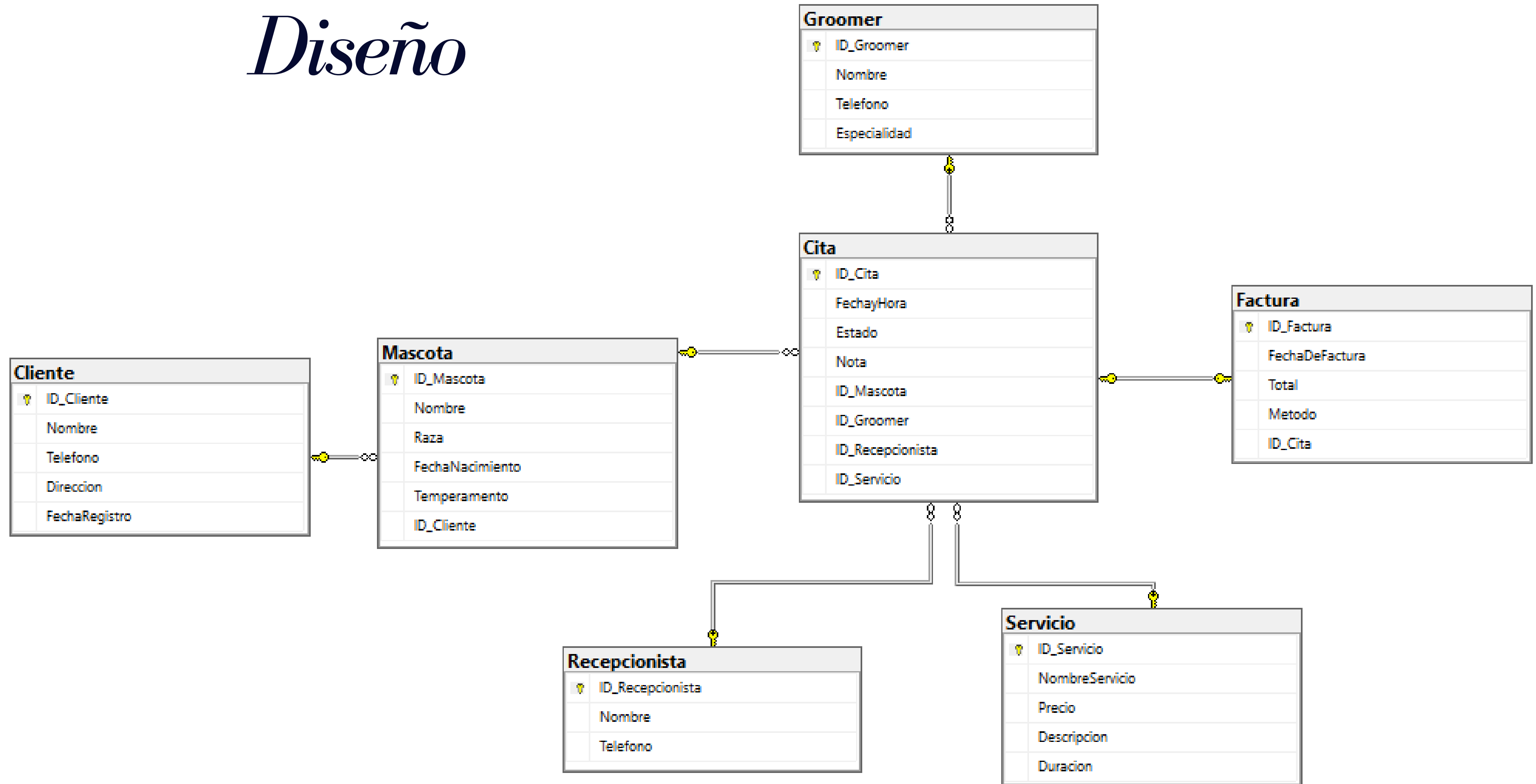
El método de pago debe pertenecer a un conjunto permitido.

Reglas de Negocio Adicionales

6. Citas Futuras

Las citas deben agendarse con una fecha futura.

Diseño



Relaciones entre Entidades



Cliente a Mascota

UN cliente puede tener MUCHAS mascotas.



Mascota a Cita

CADA mascota puede tener VARIAS citas registradas.



Groomer a Cita

UN groomer puede atender MÚLTIPLES citas.



Recepcionista a Cita

CADA cita es registrada por UN recepcionista.



Cita a Factura

UNA cita puede generar UNA única factura.



Cita a Servicio

MÚLTIPLES citas tienen UN servicio.



Scripts DDL

RESTRICCIONES DE IDENTIDAD (PK)

Reglas directas en tablas para validar datos críticos (ej. Total > 0, Estado de cita).

INTEGRIDAD REFERENCIAL (FK)

Relaciones entre entidades.

VALIDACIONES (CHECK)

Todas las tablas tienen una llave primaria con auto-incremento.

```
Create Table Recepcionista (  
  ID_Recepcionista INT IDENTITY (1,1) PRIMARY KEY,  
  Nombre VARCHAR (50) NOT NULL,  
  Telefono VARCHAR(10) NOT NULL  
)  
  
Create Table Cliente(  
  ID_Cliente INT IDENTITY (1,1) PRIMARY KEY,  
  Nombre VARCHAR (50) NOT NULL,  
  Telefono VARCHAR(10) NOT NULL,  
  Direccion VARCHAR (50) NULL,  
  FechaRegistro DATE NOT NULL DEFAULT (CAST(GETDATE() AS DATE))  
)  
  
Create Table Mascota (  
  ID_Mascota INT IDENTITY (1,1) PRIMARY KEY,  
  Nombre VARCHAR (50) NOT NULL,  
  Raza VARCHAR (50) NULL,  
  FechaNacimiento DATE NULL  
  CHECK(FechaNacimiento IS NULL OR FechaNacimiento < CAST(GETDATE() AS DATE)),  
  Temperamento VARCHAR (50) NOT NULL,  
  ID_Cliente INT NOT NULL,  
  FOREIGN KEY (ID_Cliente) REFERENCES Cliente(ID_Cliente)  
)  
  
Create Table Groomer(  
  ID_Groomer INT IDENTITY (1,1) PRIMARY KEY,  
  Nombre VARCHAR (50) NOT NULL,  
  Telefono VARCHAR(10) NULL,  
  Especialidad VARCHAR(50) NULL  
)
```

```
Create Table Cita (  
  ID_Cita INT IDENTITY(1,1) PRIMARY KEY,  
  FechayHora DATETIME NOT NULL  
  CHECK(FechayHora > CAST(GETDATE() AS DATE)),  
  Estado VARCHAR (15) NOT NULL  
  CHECK (Estado in ('Pendiente', 'Confirmada', 'Atendida', 'Cancelada')),  
  Nota VARCHAR (200) NULL,  
  ID_Mascota INT NOT NULL,  
  ID_Groomer INT NOT NULL,  
  ID_Recepcionista INT NOT NULL,  
  FOREIGN KEY (ID_Mascota) REFERENCES Mascota(ID_Mascota),  
  FOREIGN KEY (ID_Groomer) REFERENCES Groomer(ID_Groomer),  
  FOREIGN KEY (ID_Recepcionista) REFERENCES Recepcionista(ID_Recepcionista)  
)  
  
Create Table Servicio(  
  ID_Servicio INT IDENTITY (1,1) PRIMARY KEY,  
  NombreServicio VARCHAR (50) NOT NULL,  
  Precio INT NOT NULL,  
  Descripcion VARCHAR (1000) NOT NULL,  
  Duracion INT NOT NULL  
)  
  
Create Table Factura(  
  ID_Factura INT IDENTITY (1,1) PRIMARY KEY,  
  FechaDeFactura DATE NOT NULL DEFAULT (CAST(GETDATE() AS DATE)),  
  Total INT NOT NULL  
  CHECK (Total > 0),  
  Metodo VARCHAR (10) NOT NULL  
  CHECK (Metodo in ('Tarjeta', 'QR', 'Efectivo')),  
  ID_Cita INT NOT NULL UNIQUE,  
  FOREIGN KEY (ID_Cita) REFERENCES Cita(ID_Cita)  
)
```

Scripts DML

El script DML tiene como objetivo poblar la base de datos con información inicial para validar el funcionamiento del sistema y permitir la operación inmediata.

—RECEPCIONISTAS

```
INSERT INTO Recepcionista (Nombre, Telefono, Usuario, PasswordHash)
VALUES ('Jose Perez', '78812345', 'jose.perez','pepita123'),
      ('Luisa Cesperez', '72134567', 'luisa.cesperez','cepedez2435');
```

—GROOMERS

```
INSERT INTO Groomer (Nombre, Telefono, Especialidad)
VALUES ('Gabriel Ortiz', '78898989', 'Corte creativo y estilismo'),
      ('Mari Vega', '78811223', 'Baño y masajes'),
      ('Miguel Arnez', '77776665', 'Corte de pelaje largo');
```

—SERVICIOS

```
INSERT INTO Servicio (NombreServicio, Precio, Descripcion, Duracion)
VALUES ('Baño básico', 78, 'Perro Pequeño: Baño con shampoo neutro, secado y corte incluido.', 45),
      ('Baño básico', 128, 'Perro Grande: Baño con shampoo neutro, secado y corte incluido.', 68),
      ('Baño medicado', 98, 'Perro Pequeño: Baño con shampoo hipoalergénico o especial para piel sensible. Incluye corte.', 68),
      ('Baño medicado', 148, 'Perro Grande: Baño con shampoo hipoalergénico o especial para piel sensible. Incluye corte.', 98),
      ('Baño antipulgas', 188, 'Perro Pequeño: Baño con productos antiparasitarios especializados. Incluye corte.', 58),
      ('Baño antipulgas', 138, 'Perro Grande: Baño con productos antiparasitarios especializados. Incluye corte.', 78),
      ('Baño + Deslanado y cepillado profundo', 188, 'Perro Pequeño: Eliminación de nudos, pelo muerto y exceso de pelaje. Incluye corte.', 68),
      ('Baño + Deslanado y cepillado profundo', 188, 'Perro Grande: Eliminación de nudos, pelo muerto y exceso de pelaje. Incluye corte.', 118),
      ('Baño + Tratamiento hidratante de pelaje', 88, 'Perro Pequeño: Aplicación de mascarilla hidratante para mejorar brillo y textura del pelaje. Incluye corte.', 45),
      ('Baño + Tratamiento hidratante de pelaje', 158, 'Perro Grande: Aplicación de mascarilla hidratante para mejorar brillo y textura del pelaje. Incluye corte.', 68);
```

—CLIENTE

```
insert into Cliente (Nombre, Telefono, Direccion)
values ('paola', '12345678', 'zona cumavi'),
      ('adriana', '87654321', '4to anillo'),
      ('luna', '23456789', 'zona madre india'),
      ('nicol', '34567898', 'zona la ramada'),
      ('yaquelin', '89876543', 'zona tropillo'),
      ('rut', '11223344', '1er anillo'),
      ('lucia', '11222345', 'zona los pozos');
```

```
insert into Cliente (Nombre, Telefono, Direccion)
values ('david', '11335577', 'av. Pirai entre 3er y 2do anillo'),
      ('luis', '11446677', 'av. Guapay entre 2do y 3er anillo'),
      ('jose', '11558899', 'av. Mutualista entre 7mo y 8vo anillo'),
      ('leonel', '22558866', 'av. Radial 27 entre 3er y 4to anillo'),
      ('pedro', '88848822', '4to anillo entre radial 21 y av. Busch');
```

```
insert into Cliente (Nombre, Telefono, Direccion)
values ('pablo', '88552244', 'av. Paragua entre 2do y 3er anillo');
```

```
insert into Cliente (Nombre, Telefono, Direccion)
values ('maria', '77448811', 'av. Moscu entre 5to y 4to anillo');
```

```
insert into Cliente (Nombre, Telefono, Direccion)
values ('juana', '66881133', 'radial 21 entre 4to y 5to anillo'),
      ('felipe', '88119922', 'av. Busch entre 4to y 3er anillo'),
      ('juliana', '55885599', 'av. La Salle entre 2do y 3er anillo'),
      ('rene', '881928374', 'calle Bolivar entre Beni y Murillo'),
      ('narta', '65748392', 'calle Sarah entre Junin y Florida');
```

—MASCOTA

```
insert into Mascota (Nombre, Temperamento, ID_Cliente, Raza, FechaNacimiento)
values ('toby', 'expulsivo', 1, 'pug', '2025-01-01'),
      ('pepe', 'pasivo-agresivo', 2, 'pitbull', '2020-02-01'),
      ('estrella', 'pasivo', 3, 'pitbull', '2021-05-01'),
      ('sebastian', 'bravo', 4, 'pug', '2018-03-01'),
      ('juancho', 'amable', 5, 'dogo', '2015-12-25'),
      ('tongo', 'expulsivo', 6, 'chihuahua', '2025-01-01'),
      ('luciano', 'pasivo', 7, 'pitbull', '2022-03-01');
```

```
insert into Mascota (Nombre, Temperamento, ID_Cliente, Raza, FechaNacimiento)
values ('romeo', 'calmado', 8, 'san bernardo', '2024-01-01'),
      ('julian', 'agresivo', 9, 'chihuahua', '2023-11-15'),
      ('felipe', 'nervioso', 18, 'pastor aleman', '2022-09-21'),
      ('roberto', 'explosivo', 11, 'pug', '2017-10-22'),
      ('saturno', 'amable', 12, 'rottweiler', '2019-07-13');
```

```
insert into Mascota(Nombre, Temperamento, ID_Cliente, Raza, FechaNacimiento)
values ('fiorella', 'calmada', 13, 'doberman', '2020-08-15');
```

```
insert into Mascota(Nombre, Temperamento, ID_Cliente, Raza, FechaNacimiento)
values ('joaquin', 'pasivo', 14, 'cane corso', '2025-05-05');
```

```
insert into Mascota(Nombre, Temperamento, ID_Cliente, Raza, FechaNacimiento)
values ('flaco', 'calmada', 15, 'bulldog', '2015-09-25'),
      ('rosalia', 'calmada', 16, 'poodle', '2025-05-21'),
      ('rufino', 'calmada', 17, 'boxer', '2016-06-06'),
      ('choco', 'calmada', 18, 'bull terrier', '2022-01-38'),
      ('gordo', 'calmada', 19, 'chihuahua', '2025-11-11');
```

—CITA

```
insert into Cita (FechayHora, ID_Mascota, ID_Groomer, ID_Recepcionista, Estado, Nota)
values ('2025-12-26 14:00:00', 1, 1, 1, 'Confirmada', 'ninguna'),
      ('2025-12-28 16:00:00', 2, 2, 1, 'Confirmada', 'tratar con amor al animalito'),
      ('2025-12-12 17:00:00', 3, 3, 2, 'Confirmada', 'ninguna'),
      ('2025-12-27 09:00:00', 4, 2, 2, 'Cancelada', 'tener cuidado'),
      ('2025-12-12 16:00:00', 5, 1, 1, 'Confirmada', 'nada'),
      ('2025-12-28 18:00:00', 6, 2, 2, 'Atendida', 'ser amable'),
      ('2025-12-22 09:00:00', 7, 2, 2, 'Confirmada', 'ninguna');
```

```
insert into Cita (FechayHora, ID_Mascota, ID_Groomer, ID_Recepcionista, Estado, Nota)
values ('2025-12-11 14:00:00', 8, 3, 1, 'Pendiente', 'ninguna'),
      ('2025-12-19 15:00:00', 9, 1, 2, 'Confirmada', 'tener cuidado'),
      ('2025-12-21 11:00:00', 18, 2, 1, 'Pendiente', 'ser amable'),
      ('2025-12-23 08:00:00', 11, 3, 2, 'Cancelada', 'cuidarse de sus mordidas'),
      ('2025-12-26 18:00:00', 12, 1, 1, 'Confirmada', 'nada');
```

```
insert into Cita (FechayHora, ID_Mascota, ID_Groomer, ID_Recepcionista, Estado, Nota, ID_Servicio)
values ('2025-12-18 08:00:00', 13, 1, 1, 'Confirmada', 'tener cuidado con sus dientes', 18);
```

```
insert into Cita (FechayHora, ID_Mascota, ID_Groomer, ID_Recepcionista, Estado, Nota, ID_Servicio)
values ('2025-12-29 16:00:00', 14, 1, 1, 'Pendiente', 'ser amables', 18);
```

```
insert into Cita (FechayHora, ID_Mascota, ID_Groomer, ID_Recepcionista, Estado, Nota, ID_Servicio)
values ('2025-12-27 09:00:00', 18, 2, 2, 'Pendiente', 'ninguna', 5),
      ('2025-12-29 09:00:00', 19, 3, 2, 'Pendiente', 'ninguna', 5),
      ('2025-12-26 18:00:00', 28, 1, 1, 'Cancelada', 'ninguna', 4),
      ('2025-12-19 18:00:00', 21, 1, 1, 'Cancelada', 'ninguna', 18),
      ('2025-12-22 15:00:00', 22, 1, 2, 'Confirmada', 'no tenerle miedo', 1);
```

—FACTURA

```
insert into Factura (ID_Cita, Total, Metodo)
values (4, 158, 'Efectivo'),
      (5, 158, 'QR'),
      (6, 78, 'QR'),
      (7, 288, 'QR');
```

```
insert into Factura (ID_Cita, Total, Metodo)
values (8, 128, 'Efectivo'),
      (9, 78, 'QR'),
      (11, 149, 'Tarjeta');
```

```
insert into Factura (ID_Cita, Total, Metodo)
values (15, 188, 'Efectivo'),
      (16, 88, 'Tarjeta'),
      (17, 188, 'QR'),
      (18, 78, 'Tarjeta'),
      (19, 188, 'Efectivo');
```

```
insert into Factura (ID_Cita, Total, Metodo)
values (28, 158, 'Efectivo');
```

```
insert into Factura (ID_Cita, Total, Metodo)
values (21, 158, 'Efectivo'),
      (22, 138, 'QR'),
      (23, 138, 'QR'),
      (24, 148, 'QR'),
      (25, 168, 'Tarjeta'),
      (26, 78, 'Efectivo');
```

Creación De Stored Procedures

1. REGISTRAR CLIENTES

2. REGISTRAR MASCOTAS

3. REGISTRAR CITA

4. REGISTRAR FACTURA

5. ACTUALIZAR ESTADO CITA

6. LISTAR CITAS DE DIA HOY

7. VER DETALLES DE CITA

1. REGISTRAR CLIENTES

Verificacion:

Se verifica que el telefono ingresado no este duplicado

Registro:

Se registran los clientes

Resultado:

Se devuelve el id generado para poder mostrarlo al usuario

En caso de error:

Si hubo algun error que la verificacion no haya detectado se retorna ese error antes de registrar el cliente

```
ALTER PROCEDURE RegistrarCliente
    @Nombre VARCHAR(50),
    @Telefono VARCHAR(10),
    @Direccion VARCHAR(50) = NULL,
    @Resultado VARCHAR(200) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @NuevoID INT;

    BEGIN TRY
        -- Verificar si existe el teléfono
        IF EXISTS (SELECT 1 FROM Cliente WHERE Telefono = @Telefono)
        BEGIN
            SET @Resultado = 'ERROR: Ya existe un cliente con este teléfono.';
            RETURN;
        END

        -- Insertar cliente
        INSERT INTO Cliente (Nombre, Telefono, Direccion, FechaRegistro)
        VALUES (@Nombre, @Telefono, @Direccion, CAST(GETDATE() AS DATE));

        -- Obtener ID recién insertado
        SET @NuevoID = SCOPE_IDENTITY();

        -- Retornar resultado con el ID
        SET @Resultado = 'SUCCESS:ID=' + CAST(@NuevoID AS VARCHAR(50));
    END TRY
    BEGIN CATCH
        -- Captura de errores
        SET @Resultado = 'ERROR: ' + ERROR_MESSAGE();
    END CATCH
END;

EXEC dbo.RegistrarCliente
go
```

2. REGISTRAR MASCOTAS

Verificaciones:

Se verifica que el ID del cliente exista

Se verifica que la fecha de nacimiento de la mascota no sea futura

Registro:

Se registra a la mascota

Resultado:

Se devuelve el id generado para poder mostrarlo al usuario

En caso de error:

Si hubo algun error que la verificacion no haya detectado se retorna ese error antes de registrar la mascota

```
alter PROCEDURE RegistrarMascota
    @Nombre VARCHAR(50),
    @Temperamento VARCHAR(50),
    @IdCliente INT,
    @Raza VARCHAR(50) = NULL,
    @FechaNac DATE = NULL,
    @Resultado VARCHAR(200) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @NuevoID INT;

    BEGIN TRY
        -- Verificar que exista el cliente
        IF NOT EXISTS (SELECT 1 FROM Cliente WHERE Id_Cliente = @IdCliente)
        BEGIN
            SET @Resultado = 'ERROR: El cliente no existe.';
            RETURN;
        END

        -- Verificar fecha de nacimiento válida
        IF @FechaNac IS NOT NULL AND @FechaNac >= CAST(GETDATE() AS DATE)
        BEGIN
            SET @Resultado = 'ERROR: La fecha de nacimiento no puede ser futura ni de hoy.';
            RETURN;
        END

        -- Insertar mascota
        INSERT INTO Mascota (Nombre, Temperamento, Id_Cliente, Raza, FechaNacimiento)
        VALUES (@Nombre, @Temperamento, @IdCliente, @Raza, @FechaNac);

        -- Obtener ID recién insertado
        SET @NuevoID = SCOPE_IDENTITY();

        -- Retornar resultado
        SET @Resultado = 'OK ID=' + CAST(@NuevoID AS VARCHAR(50));
    END TRY
    BEGIN CATCH
        SET @Resultado = 'ERROR: ' + ERROR_MESSAGE();
    END CATCH
END;
```

3. REGISTRAR CITA

Verificaciones:

- Se verifica:
- Macota, groomer, recepcionista existentes
 - Fecha y hora de la cita sean futuras
 - El estado de la cita no sea cancelada o atendida
 - El id del servicio sea valido
 - No hayan citas ya existentes en esa fecha/hora con el mismo groomer

Registro:

Se registra la cita

Resultado:

Se devuelve el id generado para poder mostrarlo al usuario

En caso de error:

Si hubo algun error que las verificaciones no hayan detectado se retorna ese error antes de registrar la cita

```
alter PROCEDURE RegistrarCita
    @FechaHora DATETIME,
    @IdMascota INT,
    @IdGroomer INT,
    @IdRecepcionista INT,
    @IdServicio INT,
    @Estado VARCHAR(15) = 'Pendiente',
    @Nota VARCHAR(200) = NULL,
    @Resultado VARCHAR(200) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @NuevoID INT;
    DECLARE @CitaConflicto INT;

    BEGIN TRY
        -- Validaciones básicas
        IF NOT EXISTS (SELECT 1 FROM Mascota WHERE Id_Mascota = @IdMascota)
        BEGIN
            SET @Resultado = 'ERROR: La mascota no existe.';
            RETURN;
        END

        IF NOT EXISTS (SELECT 1 FROM Groomer WHERE Id_Groomer = @IdGroomer)
        BEGIN
            SET @Resultado = 'ERROR: El groomer no existe.';
            RETURN;
        END

        IF NOT EXISTS (SELECT 1 FROM Recepcionista WHERE Id_Recepcionista = @IdRecepcionista)
        BEGIN
            SET @Resultado = 'ERROR: La recepcionista no existe.';
            RETURN;
        END

        IF @FechaHora <= GETDATE()
        BEGIN
            SET @Resultado = 'ERROR: La fecha debe ser futura.';
            RETURN;
        END

        -- Solo permitir Pendiente o Confirmada
        IF @Estado NOT IN ('Pendiente','Confirmada')
        BEGIN
            SET @Resultado = 'ERROR: No se puede registrar una cita con estado Atendida o Cancelada.';
            RETURN;
        END

        -- Validar ID de servicio
        IF @IdServicio NOT BETWEEN 1 AND 10
        BEGIN
            SET @Resultado = 'ERROR: ID del servicio no válido.';
            RETURN;
        END

        -- Validación de cita solapada: mismo groomer y misma fecha/hora
        SELECT TOP 1 @CitaConflicto = Id_Cita
        FROM Cita
        WHERE Id_Groomer = @IdGroomer
        AND FechaYHora = @FechaHora
        AND Estado IN ('Pendiente', 'Confirmada');

        IF @CitaConflicto IS NOT NULL
        BEGIN
            SET @Resultado = 'ERROR: Ya existe una cita pendiente o confirmada para este groomer a esa hora.';
            RETURN;
        END

        -- Insertar la cita
        INSERT INTO Cita (FechaYHora, Estado, Nota, Id_Mascota, Id_Groomer, Id_Recepcionista, Id_Servicio)
        VALUES (@FechaHora, @Estado, @Nota, @IdMascota, @IdGroomer, @IdRecepcionista, @IdServicio);

        SET @NuevoID = SCOPE_IDENTITY();

        -- Devolver resultado con ID
        SET @Resultado = 'OK ID=' + CAST(@NuevoID AS VARCHAR(50));

    END TRY
    BEGIN CATCH
        SET @Resultado = 'ERROR: ' + ERROR_MESSAGE();
    END CATCH

    EXEC dbo.RegistrarCita
go
```

4. REGISTRAR FATURA

Verificaciones:

Se verifica:

- Id de la cita debe existir
- Id de la cita no debe tener facturas ya registradas
- El total a pagar no debe ser menor o igual a cero
- El metodo de pago debe ser valido (Tarjeta, Qr, Efectivo)

Registro:

Se registra la factura

Resultado:

Se devuelve un mensaje de aprobacion

En caso de error:

Si hubo algun error que las verificaciones no hayan detectado se retorna ese error antes de registrar la factura

```
alter PROCEDURE RegistrarFactura
    @IdCita INT,
    @Total INT,
    @Metodo VARCHAR(20),
    @Resultado VARCHAR(200) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        -- Validar que exista la cita
        IF NOT EXISTS (SELECT 1 FROM Cita WHERE Id_Cita = @IdCita)
        BEGIN
            SET @Resultado = 'ERROR: La cita no existe.';
            RETURN;
        END

        -- Validar que no exista factura para esa cita
        IF EXISTS (SELECT 1 FROM Factura WHERE Id_Cita = @IdCita)
        BEGIN
            SET @Resultado = 'ERROR: Ya existe una factura para esta cita.';
            RETURN;
        END

        -- Validar total
        IF @Total <= 0
        BEGIN
            SET @Resultado = 'ERROR: El total debe ser mayor a 0.';
            RETURN;
        END

        -- Validar método de pago
        IF UPPER(@Metodo) NOT IN ('TARJETA', 'QR', 'EFECTIVO')
        BEGIN
            SET @Resultado = 'ERROR: Método de pago no permitido.';
            RETURN;
        END

        -- Insertar factura
        INSERT INTO Factura (Total, Metodo, Id_Cita)
        VALUES (@Total, @Metodo, @IdCita);

        SET @Resultado = 'OK';

    END TRY
    BEGIN CATCH
        SET @Resultado = 'ERROR: ' + ERROR_MESSAGE();
    END CATCH
END;

EXEC dbo.RegistrarFactura
go
```

5. ACTUALIZAR ESTADO CITA

Verificaciones:

Se verifica:

- Id de la cita debe existir
- El nuevo estado debe ser valido (Pendiente, Confirmada, Atendida, Cancelada)

Actualizacion:

Se actualiza el estado de la cita

Resultado:

Se devuelve un mensaje de aprobacion

En caso de error:

Si hubo algun error que las verificaciones no hayan detectado se retorna ese error antes de actualizar la cita

```
alter PROCEDURE ActualizarEstadoCita
    @IdCita INT,
    @Estado VARCHAR(15),
    @Resultado VARCHAR(200) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        -- Validar que exista la cita
        IF NOT EXISTS (SELECT 1 FROM Cita WHERE Id_Cita = @IdCita)
        BEGIN
            SET @Resultado = 'ERROR: La cita no existe.';
            RETURN;
        END

        -- Validar estado permitido
        IF @Estado NOT IN ('Pendiente', 'Confirmada', 'Atendida', 'Cancelada')
        BEGIN
            SET @Resultado = 'ERROR: Estado no válido.';
            RETURN;
        END

        -- Actualizar estado de la cita
        UPDATE Cita
        SET Estado = @Estado
        WHERE Id_Cita = @IdCita;

        SET @Resultado = 'OK';

    END TRY
    BEGIN CATCH
        SET @Resultado = 'ERROR: ' + ERROR_MESSAGE();
    END CATCH
END;

EXEC dbo.ActualizarEstadoCita
go
```

6. LISTAR CITAS DE DIA HOY

Verificacion:

Se verifica que la fecha de la cita sea igual a la actual

Resultado:

Se muestra:

- El id de la cita
- Fecha y hora de la cita
- Estado de la cita
- Nombre del cliente
- Nombre de la mascota
- Nombre del groomer

Se ordenan los resultados ascendentemente en base a la fecha y hora de cada cita

```
alter PROCEDURE ListarCitasDeDiaHoy
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        c.Id_Cita AS id_cita,
        c.FechaYHora AS fechahora,
        c.Estado AS estado,
        cl.Nombre AS cliente,
        m.Nombre AS mascota,
        g.Nombre AS groomer
    FROM Cita c
    INNER JOIN Mascota m ON c.Id_Mascota = m.Id_Mascota
    INNER JOIN Cliente cl ON m.Id_Cliente = cl.Id_Cliente
    INNER JOIN Groomer g ON c.Id_Groomer = g.Id_Groomer
    WHERE CAST(c.FechaYHora AS DATE) = CAST(GETDATE() AS DATE)
    ORDER BY c.FechaYHora;
END;

EXEC dbo.ListarCitasDeDiaHoy
go
```

7. VER DETALLES DE CITA

Verificacion:

Se verifica que la cita exista

Se hace INNER JOIN de las tablas:

- Cita
- Mascota
- Cliente
- Groomer

Se hace un LEFT JOIN con la tabla servicio para mostrar todos los servicios que la cita hizo

Resultado:

Se muestra:

- Id de la cita
- Fecha y hora de la cita
- Nota de la cita
- Nombre del cliente
- Nombre de la mascota
- Nombre del groomer
- Nombre del servicio, si no existe se muestra sin servicio
- Precio del servicio, si no existe se muestra 0

```
ALTER PROCEDURE VerDetallesDeCita
    @IdCita INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Validar si existe la cita
    IF NOT EXISTS (SELECT 1 FROM Cita WHERE Id_Cita = @IdCita)
    BEGIN
        RAISERROR ('ERROR: No existe una cita con ID %d.', 16, 1, @IdCita);
        RETURN;
    END

    -- Retornar datos de la cita
    SELECT
        c.Id_Cita AS cita_id,
        c.FechaYHora AS fecha_hora,
        c.Estado AS estado_cita,
        c.Nota AS nota_cita,
        cl.Nombre AS nombre_cliente,
        m.Nombre AS nombre_mascota,
        g.Nombre AS nombre_groomer,
        ISNULL(s.NombreServicio, 'Sin servicio') AS nombre_servicio,
        ISNULL(s.Precio, 0) AS precio_servicio
    FROM Cita c
    INNER JOIN Mascota m ON c.Id_Mascota = m.Id_Mascota
    INNER JOIN Cliente cl ON m.Id_Cliente = cl.Id_Cliente
    INNER JOIN Groomer g ON c.Id_Groomer = g.Id_Groomer
    LEFT JOIN Servicio s ON c.Id_Servicio = s.Id_Servicio
    WHERE c.Id_Cita = @IdCita;
END;

EXEC dbo.VerDetallesDeCita
```

¡Muchas gracias
por la atención!

